DA3406 Assignment 2

Prabhav Gupta DA24B018

Database and Changes made to A1

Scale up

I used seed 24018 and `SCALE = 50`. The database is about 4 GB.

Table	Rows	Table	Rows
AppUser	250,000	Impression	15,400,863
Video	1,000,000	ViewSegment	6,732,140
AudioTrack	45,000	AgentSession	100,000
ModerationDecision	2,345,054	AgentTurn	259,760

The other tables are roughly 50 times bigger than in A1. The small reference tables, such as `ModelPrice` and `LlmModel`, stay the same.

Fixing A1 damage

- In A1 the generator did not scale the audio tracks and the follows with `SCALE`. I changed this, so they also grow 50 times.
- At this size the generator needed too much memory (roughly 12 GB). I changed how the impressions are stored and inserted, so it needs about half. The data it creates is the same as before (checked on small runs).

Task 1: Access Paths & Cost Arithmetic

I used F12, F10 and F1. F12 and F10 join the biggest tables. The plans were taken before I added any index, and I only show the lines for the real tables. The full plans are in Appendix A.

1.1

F12

```
SCAN s
SEARCH i USING INDEX ix_impression_user (user_id=?)
SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
SEARCH vs USING INDEX sqlite_autoindex_ViewSegment_1 (impression_id=?)
SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
```

F10 (here `s` is ViewSegment)

```
SCAN s
SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
```

F1

```
SEARCH Video
SCAN t
SEARCH v USING INDEX ix_video_track (audio_track_id=?)
```

Query	Table	Access path
F12	AgentSession	full scan
	Impression	existing index `ix_impression_user`
	Recommendation	primary-key lookup
	ViewSegment	existing index (the primary key's index)
	AgentTurn	primary-key lookup
F10	ViewSegment	full scan
	Impression	primary-key lookup
	Video	primary-key lookup
F1	AudioTrack	full scan
	Video	existing index `ix_video_track`
	Video (`MAX` step)	full scan in practice, because there is no index on `uploaded_at`

In the plans, a `?` is a value that the outer loop supplies for each row while the query runs. F10 and F1 have more plan lines (temporary results, sorting), but they do not touch other tables.

1.2

- Rows: $N = 6{,}732{,}140$.
- Block size: $B = 4{,}096$ bytes, which is the page size of the database.
- Row size: about $R = 43$ bytes. This is my estimate from the columns (a 20-character timestamp, a few small numbers and the row overhead). I could not measure it.
- Rows per block: $\lfloor 4096/43 \rfloor = 95$.
- Blocks: $\lceil 6{,}732{,}140/95 \rceil = 70{,}865$, which is about 290 MB.

1.3

F12. It starts from AgentSession and not from Recommendation. The statistics say that `ix_impression_rec` gives 39 rows per key, so starting there looks expensive. That number is wrong (see 3.3).

F10. Nothing is filtered, so all of ViewSegment is needed. Starting from ViewSegment (6.7M rows) is cheaper than from Impression (15.4M rows), because then every row needs just one lookup in Impression and one in Video.

F1. It scans the small AudioTrack table and uses `ix_video_track` for the videos. The date filter cannot be used for the index, because `CROSS JOIN a` makes the date known only at the end. This matters in Task 2.

Task 2: Composite Index Design

2.1

```
CREATE INDEX idx_video_track_uploaded ON Video(audio_track_id, uploaded_at);
CREATE INDEX idx_imp_rec_user_p ON Impression(recommendation_id, user_id)
  WHERE recommendation_id IS NOT NULL;
CREATE INDEX idx_turn_model_created ON AgentTurn(model_id, created_at);
```

F1. `audio_track_id` serves the join clause (an equality). `uploaded_at` serves the `WHERE` clause (a range). The equality column goes first and the range column second. If the order is reversed, the range column leads and the join cannot use the index.

F12. Both columns serve the join clause, both as equalities, so with both values known either order works. `recommendation_id` goes first because it is almost unique and the join

gets it first. If `user_id` came first, a lookup that only knows `recommendation_id` could not use the index. The index is partial: only 400,863 of the 15.4 million impressions have a recommendation, and the NULLs never match this join.

F7. `model_id` serves the join clause with ModelPrice (an equality). `created_at` serves the same clause, where it is compared with the price dates (a range), and the `WHERE` filter. The equality goes first. If the order is reversed, `model_id` comes after a range column and cannot be used for the seek.

2.2

Each index was built on a copy of the same database. The times are medians of 3 to 6 runs.

F1.

Query	Before	After
F1 as written	7.5 s	0.57 s
F1 rewritten	2.5 s	0.10 s
F1 rewritten, reversed index	2.5 s	3.2 s

```
before:
SEARCH v USING INDEX ix_video_track (audio_track_id=?)
after, F1 as written:
SEARCH v USING COVERING INDEX idx_video_track_uploaded (audio_track_id=?)
after, F1 rewritten:
SEARCH v USING COVERING INDEX idx_video_track_uploaded (audio_track_id=? AND uploaded_at>?)
```

I rewrote F1 (`F1_rewritten.sql`) because `CROSS JOIN a` hides the date from the index. In the rewrite the date is worked out once in a subquery. It gives the same rows. F1 as written is still faster with the index, because the index covers the query. The reversed index is not used by the join.

F12.

	Before	After
Time	about 239 s	about 1 s
Plan starts with	`SCAN s` (AgentSession), then `ix_impression_user`	`SCAN i USING COVERING INDEX idx_imp_rec_user_p`

The plan changes completely. It now starts from the small partial index. The same plan took about 38 s in an earlier run and 239 s on the copy. I do not know exactly why, probably it depends on what the computer had in memory. So the gain is somewhere between 40 and 250 times.

F7. This index makes the query slower: 0.32 s before, 0.71 s with the index.

```
before:  SCAN t USING INDEX sqlite_autoindex_AgentTurn_1
         SEARCH u USING INTEGER PRIMARY KEY
         SCAN mp
after:   SCAN mp
         SEARCH t USING INDEX idx_turn_model_created (model_id=? AND created_at>?)
         SEARCH u USING INTEGER PRIMARY KEY
```

The planner starts from ModelPrice and uses the index. But the date in the seek is the start of a price period, not the last-month limit, so it does not narrow the search. The index also does not cover the query, so every entry needs a lookup in AgentTurn. The real fix for F7 is in Task 4.

2.3

I rejected `ModerationDecision(to_state)`. F2 filters on `to_state = 'live'`. But `to_state` has only five values, and about 41% of the rows are `live`. Following that many index entries costs more than a scan. Also, F2 needs the latest decision of each video, which `ix_moddec_video` already handles. And the index would slow down inserts.

Task 3: Join Algorithms

3.1

The engine only uses nested loops. It has no hash join and no sort-merge join.

- **F10.** ViewSegment is the outer loop. For each row it looks up Impression and then Video by primary key. This is an index nested-loop join.
- **F12.** AgentSession is the outer loop. Then it uses `ix_impression_user` for Impression and lookups for the other tables. This is also an index nested loop, but each Impression lookup returns about 62 rows.

3.2

I take the join of ViewSegment r (outer) and Impression s (inner) from F10. The cost is the number of block transfers. With the block model of Task 1.2: $n_r = 6{,}732{,}140$ and $b_r = 70{,}865$. An Impression row is about 55 bytes, so $n_s = 15{,}400{,}863$ and $b_s = 208{,}120$. I assume $M = 500$ memory pages.

Strategy	Formula	Block transfers
Nested loop	$n_r \times b_s + b_r$	1.4×10^{12}
Block nested loop	$b_r + \lceil b_r/(M-2) \rceil \times b_s$	29.8 million
Indexed nested loop	$b_r + n_r \times c$, with $c = 4$	27.0 million
Sort-merge	$b_r + b_s$	0.28 million
Hash join	$3(b_r + b_s) + 4\,n_h$, $n_h = 143$	0.84 million

Here $c = 4$ is the height of the B-tree of Impression, which is a worst case, and $n_h = \lceil b_r/(M-2) \rceil = 143$. For sort-merge I assume that both tables are already sorted on `impression_id`. This is true for this database: Impression is stored by that id, and the generator inserts the segments of an impression right after it. If a sort were needed, it would add about 0.84 million.

For the hash join, n_h is the number of partitions, chosen so that each partition of ViewSegment fits in memory. The term $3(b_r + b_s)$ is one read to split both tables into partitions, one write of the partitions and one read to join them. The term $4\,n_h$ is a small extra for the partly filled block at the end of each partition (written and read again, for both tables).

The cheapest is sort-merge, then hash join. The engine does not agree with this, and it cannot, because it has neither of them. It uses the indexed nested loop, which is the cheapest of the nested-loop types.

3.3 Challenge

I use AgentSession, Recommendation and Impression. (AgentTurn is only a lookup between them, and ViewSegment is joined last to the few impressions that are left.) AgentSession has 100,000 rows and Recommendation has 643,155. Only 400,863 impressions have a recommendation, and there are about 62 impressions per user.

My order starts with Recommendation, then its session, then Impression. That gives 643,155 rows and then at most 400,863. **The engine's order** starts with the 100,000 sessions. Joined with Impression this gives about $100{,}000 \times 62 = 6.2$ million rows, and most of them are thrown away afterwards. That is about 10 times more than my biggest result.

I forced my order with `CROSS JOIN` (`F12_forced.sql`). It returns the same rows.

Order	Time
Engine's (AgentSession first)	38 s to 239 s
Mine (Recommendation first)	0.5 s

So my order is right and the engine is wrong here. The reason is the statistics. They say 39 rows per key for `ix_impression_rec`, but the real value is 1. The NULLs are counted as one big key: 15.4M/400,864 $\approx$ 39. So the engine thinks each lookup is expensive and avoids my order. The partial index from Task 2 fixes this.

Task 4: Predicate Pushdown

4.1

I use F7 (session cost for the last month). Its filter on `created_at` uses only AgentTurn, but in the query it is applied after the joins. Let c be the cutoff date and θ the price-period condition.

As written:

$$\gamma\Big(\sigma_{\text{created_at}\geq c}\big((\text{AgentTurn} \bowtie \text{TurnUsage}) \bowtie_\theta \text{ModelPrice}\big)\Big)$$

Pushed down:

$$\gamma\Big(\big(\sigma_{\text{created_at}\geq c}(\text{AgentTurn}) \bowtie \text{TurnUsage}\big) \bowtie_\theta \text{ModelPrice}\Big)$$

The rule is $\sigma_p(R \bowtie S) \equiv \sigma_p(R) \bowtie S$ when p only uses attributes of R. Here p only uses AgentTurn.

The rule does not hold for a filter on the other side of an outer join. In $R \bowtie_L S$ a filter on S applied after the join removes the rows where S is NULL. If S is filtered first, those rows stay. F7 has only inner joins, so the rewrite is safe here.

4.2

As written (F7 in `queries.sql`):

```
WITH pj AS (
  SELECT t.turn_id, t.session_id, t.template_version_id, t.created_at,
         u.input_tokens, u.output_tokens, u.cached_tokens,
         ROUND((u.input_tokens - u.cached_tokens)/1000.0*mp.input_rate
               + u.cached_tokens/1000.0*mp.cached_rate
               + u.output_tokens/1000.0*mp.output_rate, 8) cost_usd
  FROM AgentTurn t
  JOIN TurnUsage u ON u.turn_id = t.turn_id
  JOIN ModelPrice mp ON mp.model_id = t.model_id
       AND t.created_at >= mp.valid_from
       AND (mp.valid_to IS NULL OR t.created_at < mp.valid_to)
),
a AS (SELECT MAX(created_at) t FROM AgentTurn)
SELECT session_id, template_version_id, COUNT(*) turns,
       SUM(input_tokens) input_tokens, SUM(output_tokens) output_tokens,
       ROUND(SUM(cost_usd), 6) session_cost_usd
FROM pj CROSS JOIN a
WHERE created_at >= strftime('%Y-%m-%dT%H:%M:%SZ', a.t, '-1 month')
GROUP BY session_id, template_version_id
HAVING SUM(cost_usd) > 0.0008
ORDER BY session_cost_usd DESC
LIMIT 15;
```

Pushed down (`F7_pushed.sql`):

```
WITH a AS (SELECT strftime('%Y-%m-%dT%H:%M:%SZ', MAX(created_at), '-1 month') cut
           FROM AgentTurn),
pj AS (
  SELECT t.turn_id, t.session_id, t.template_version_id, t.created_at,
         u.input_tokens, u.output_tokens, u.cached_tokens,
         ROUND((u.input_tokens - u.cached_tokens)/1000.0*mp.input_rate
               + u.cached_tokens/1000.0*mp.cached_rate
               + u.output_tokens/1000.0*mp.output_rate, 8) cost_usd
  FROM AgentTurn t
  JOIN TurnUsage u ON u.turn_id = t.turn_id
  JOIN ModelPrice mp ON mp.model_id = t.model_id
       AND t.created_at >= mp.valid_from
       AND (mp.valid_to IS NULL OR t.created_at < mp.valid_to)
  WHERE t.created_at >= (SELECT cut FROM a))
SELECT session_id, template_version_id, COUNT(*) turns,
       SUM(input_tokens) input_tokens, SUM(output_tokens) output_tokens,
       ROUND(SUM(cost_usd), 6) session_cost_usd
FROM pj
GROUP BY session_id, template_version_id
HAVING SUM(cost_usd) > 0.0008
ORDER BY session_cost_usd DESC
LIMIT 15;
```

Both give the same 15 rows. As written it takes 0.34 s. Pushed down it takes 0.11 s, which is about 3 times faster. The engine does not do this itself. In the plan as written it joins the three tables first and reaches the cutoff last.

Task 5: Concurrency: Schedules, Anomalies, Isolation

5.1

In T2, connection A inserts a moderation decision for video 1 and does not commit. B reads the state of the video. Then A rolls back. Let x be the state of video 1.

```
A: BEGIN IMMEDIATE
A: INSERT decision ('taken_down')          w_A(x)
B: SELECT current_state -> 'live'          r_B(x)
A: ROLLBACK
```

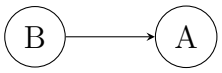
A rolled back, so it is not part of the committed schedule. Only B is left, so the graph has no edges and the schedule is trivially conflict-serializable.

If I treat A as committed, the only conflict is $(w_A(x), r_B(x))$. B read the older committed version, so B comes before A. This is a multiversion dependency (a reader of the older version comes before the writer), and it gives the edge $B \to A$. There is no cycle, and the equivalent

serial order is B, then A. (By the clock A wrote first, but B did not see that write, so the edge is not $A \to B$.)



committed schedule (A rolled back)



if A is treated as committed

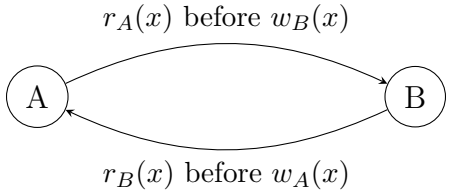
5.2

The same like is retracted from two devices at once. Each one reads the like (still active), then writes `retracted_at` in `VideoLike` and adds a `like_retracted` row in `EngagementSignal`. Let x be the like with `like_id = 8`.

A and B are the two logical transactions of the application. Each one is a read, a decision and the writes. The database does not see them as single transactions here: the read is a statement of its own, and only the writes are inside `BEGIN` and `COMMIT`. The graph below is for the logical transactions.

```
A: r(x)   B: r(x)   A: w(x), commit   B: w(x), commit
```

$r_A(x)$ before $w_B(x)$ gives $A \to B$, and $r_B(x)$ before $w_A(x)$ gives $B \to A$. There is a cycle, so it is not conflict-serializable. It is a lost update: B overwrites the time written by A.



In the demo the like ends up with two retraction rows. A serial run could never do this, because the second transaction would see the like already retracted and do nothing. In ScrollSense this counts the retraction twice in `net_likes`.

5.3

I ran this on a copy of the database with two connections, tab A and tab B (see `concurrency_demo.sql`).

Without the fix, both tabs read the like (each read is one statement, outside a transaction) and both retract it. The result is two retraction rows and the time of B. The read and the writes of one tab are not one database transaction, but together they are the logical transaction of 5.2.

With the fix, each transaction starts with `BEGIN IMMEDIATE`, so it takes the write lock first. This puts the read, the decision and the writes into one database transaction. Tab A gets the lock. Tab B waits about 3 seconds and then fails with `database is locked`. Tab A retracts the like and commits. Tab B tries again, sees that the like is already retracted, and does nothing. The result is one retraction row.

Cost. While tab A holds the write lock, every other writer waits, even one that wants a different row. If the wait is too long the writer fails with `database is locked`, and the application has to retry.

Appendix A: Full Plans

Task 1

F12

```
SCAN s
SEARCH i USING INDEX ix_impression_user (user_id=?)
SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
SEARCH vs USING INDEX sqlite_autoindex_ViewSegment_1 (impression_id=?)
SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY
```

F10 (here `s` is ViewSegment)

```
CO-ROUTINE (subquery-5)
  CO-ROUTINE (subquery-6)
    MATERIALIZE r
      CO-ROUTINE (subquery-7)
        MATERIALIZE d
          SCAN s
          SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
          SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
          USE TEMP B-TREE FOR GROUP BY
        SCAN d
        USE TEMP B-TREE FOR ORDER BY
      SCAN (subquery-7)
    MATERIALIZE a
      SEARCH d
    SCAN a
    SCAN r
    BLOOM FILTER ON p (creator_id=? AND day=?)
    SEARCH p USING AUTOMATIC COVERING INDEX (creator_id=? AND day=?) LEFT-JOIN
    USE TEMP B-TREE FOR ORDER BY
  SCAN (subquery-6)
SCAN (subquery-5)
USE TEMP B-TREE FOR ORDER BY
```

F1

```
MATERIALIZE a
  SEARCH Video
SCAN t
SEARCH v USING INDEX ix_video_track (audio_track_id=?)
SCAN a
USE TEMP B-TREE FOR ORDER BY
```

Task 2

The plans before the index for F1 as written and for F12 are the Task 1 plans above.

F1 as written, with the index

```
MATERIALIZE a
  SEARCH Video USING COVERING INDEX idx_video_track_uploaded
SCAN t
SEARCH v USING COVERING INDEX idx_video_track_uploaded (audio_track_id=?)
SCAN a
USE TEMP B-TREE FOR count(DISTINCT)
USE TEMP B-TREE FOR ORDER BY
```

F1 rewritten, before

```
SCAN t
SEARCH v USING INDEX ix_video_track (audio_track_id=?)
SCALAR SUBQUERY 1
  SEARCH Video
USE TEMP B-TREE FOR ORDER BY
```

F1 rewritten, with the index

```
SCAN t
SEARCH v USING COVERING INDEX idx_video_track_uploaded (audio_track_id=? AND uploaded_at>?)
SCALAR SUBQUERY 1
  SEARCH Video USING COVERING INDEX idx_video_track_uploaded
USE TEMP B-TREE FOR count(DISTINCT)
USE TEMP B-TREE FOR ORDER BY
```

F1 rewritten, with the reversed index

```
SCAN t
SEARCH v USING INDEX ix_video_track (audio_track_id=?)
SCALAR SUBQUERY 1
  SEARCH Video USING COVERING INDEX idx_video_up_track
USE TEMP B-TREE FOR ORDER BY
```

F12, with the index

```
SCAN i USING COVERING INDEX idx_imp_rec_user_p
SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
SEARCH vs USING INDEX sqlite_autoindex_ViewSegment_1 (impression_id=?)
SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY
```

F7, before

```
MATERIALIZE a
  SEARCH AgentTurn USING COVERING INDEX ix_turn_created
SCAN t USING INDEX sqlite_autoindex_AgentTurn_1
SEARCH u USING INTEGER PRIMARY KEY (rowid=?)
SCAN mp
SCAN a
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY
```

F7, with the index

```
MATERIALIZE a
  SEARCH AgentTurn USING COVERING INDEX ix_turn_created
SCAN mp
SEARCH t USING INDEX idx_turn_model_created (model_id=? AND created_at>?)
SEARCH u USING INTEGER PRIMARY KEY (rowid=?)
SCAN a
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY
```

Task 3.3 and Task 4

F12, order forced to start with Recommendation

```
SCAN r USING INDEX sqlite_autoindex_Recommendation_2
SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
SEARCH i USING INDEX ix_impression_rec (recommendation_id=?)
SEARCH vs USING INDEX sqlite_autoindex_ViewSegment_1 (impression_id=?)
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY
```

F7, filter pushed down (F7 as written has the same plan as "F7, before" above)

```
SCAN t USING INDEX sqlite_autoindex_AgentTurn_1
SCALAR SUBQUERY 2
  CO-ROUTINE a
    SEARCH AgentTurn USING COVERING INDEX ix_turn_created
  SCAN a
SEARCH u USING INTEGER PRIMARY KEY (rowid=?)
SCAN mp
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY
```

Appendix B: Use of an LLM

I used Claude to help with checking my plan, writing scripts and the generator changes, and drafting this report. I ran all the commands and experiments myself.

Some of its advice was wrong. For the concurrency demo its first setup did not give two separate connections, and tab B failed with "cannot start a transaction within a transaction". I created a second connection and then the demo worked.